BITACORA TECNICA

PWM automatico para la fan de la AXU3EGB

Vivado 2026.1 + AMD EDF 26.06

Sesion	24 de octubre de 2026
Placa	ALINX AXU3EGB, Zynq UltraScale+ ZU3EG
Linux	AMD EDF 26.06, kernel 6.18.10-xilinx
Resultado	Arranque y apagado ambiental comprobados; rampa pendiente de validar

Esta bitacora recupera la parte que se automatizo cuando dejamos de avanzar paso a paso. Incluye decisiones, codigos completos, comandos, salidas compartidas, estado real y pendientes. No inventa salidas que no quedaron guardadas.

Repositorio: https://github.com/abelwu2001/AXU3EGB_EDF-AMD_VIVADO-2026.2_testing

1. Resumen: que se hizo

Si, fue bastante: se integro el PWM AXI al diseño de Vivado 2026.1; se genero un BOOT.BIN para AMD EDF 26.06 con hardware actualizado; se agrego la descripcion de PL a Linux mediante overlay; se registro el mapa con UIO; y se instalo un servicio que lee temperatura y ordena duty cycle.

Parte	Resultado
Vivado	PS minimo + axi_pwm; base AXI 0xA0000000; salida AA11.
Arranque EDF	BOOT.BIN con FSBL, PMU FW, bitstream, ATF, DTB y U-Boot.
Linux hardware	Overlay y UIO; uio4 mapeado en 0xA0000000.
Servicio	fan-pwm.service habilitado/activo; PID 797 en la sesion compartida.
Prueba real	A 29-30 C, DUTY=0 y la fan se detuvo fisicamente.
Falta	Desplegar el ultimo Python y probar 30% y curva 40-60 C.

Que fue automatico y que hiciste vos

Yo prepare los archivos, build de BOOT.BIN, scripts, revision/documentacion y push a GitHub. Vos mantuviste placa conectada por JTAG/UART/SSH, copiaste archivos y ejecutaste los comandos de instalacion, compartiendo las respuestas. La placa no se programa automaticamente: el BOOT.BIN se copia a la microSD y se reinicia; luego systemd inicia el software.

Dos versiones

- La placa ejecuta el Python instalado en /usr/local/sbin. Esa version aplica el overlay, lee sensores y apaga la fan en reposo.

- GitHub incluye un Python actualizado que tambien da un kick de 2 s cuando la curva vuelve a encender la fan. Esa version aun no esta copiada en la placa.

- El minimo real de 30% y la rampa no se probaron fisicamente. Linux informa el duty pedido, no RPM: no hay tacometro conectado.

2. Del Constant al PWM

La prueba con Constant mostro que 0 hacia girar la fan al maximo y 1 la apagaba. La entrada se comporta activa en bajo. Constant solo fija 0 o 1; PWM genera pulsos con duty variable.

Se reutilizo la IP axi_pwm del proyecto Vivado 2023.2 como referencia para la base minimalista 2026.1. No se migro todo el TRD. El PS ejecuta Linux; la PL contiene el contador PWM.

Bloque	En simple
PS Zynq	Cortex-A53, Linux, memoria/controladores.
PL	Logica FPGA que genera pulsos PWM.
AXI SmartConnect	Interconecta master PS y periféricos AXI.
axi_pwm / S00_AXI	Registros que reciben CTRL, PERIOD y DUTY.
rst_ps_99M	Reset sincronizado al reloj PL.
Address Editor	Asigna rango de direcciones a la IP.
fan_on_n	Puerto logico hacia pad AA11.

Se activo AXI HPM0 FPD (master PS hacia PL); antes no habia conexion AXI valida para S00_AXI. Address Editor asigno S00_AXI_reg en 0xA0000000, rango 64K, fin 0xA000FFFF. Es direccion AXI en memoria, no SPI.

XDC: constraints

PACKAGE_PIN asigna el puerto al pin fisico; IOSTANDARD informa el nivel electrico; COMPRESS comprime el bitstream. XDC no crea la onda PWM: lo hace HDL.

```
set_property PACKAGE_PIN AA11 [get_ports fan_on_n]
set_property IOSTANDARD LVCMOS33 [get_ports fan_on_n]
```

```
set_property BITSTREAM.GENERAL.COMPRESS TRUE [current_design]
```

Offset	Registro	Funcion
0x00	CTRL	bit 0 habilita; bit 1 invierte; CTRL=3 activa ambos.
0x04	PERIOD	Ciclos por periodo; 4000 a 99.999 MHz ~=25 kHz.
0x08	DUTY	Ciclos activos; 1200/4000=30%; 4000/4000=100%.

Se valido diseño, Generate Output Products, sintesis, implementacion y bitstream. La validacion dijo sin errores ni critical warnings; synthesis e implementation completaron y failed routes fue 0.

HDL del contador PWM

Fuente completa: hardware/fan-control-stage2/ip_repo/axi_pwm_1_0/hdl/pwm_core.vhd

```
-- hdl/pwm_core.vhd
library ieee; use ieee.std_logic_1164.all; use ieee.numeric_std.all;
entity pwm_core is
  port (
    clk       : in  std_logic;
    rst_n     : in  std_logic;
    enable    : in  std_logic;
    polarity  : in  std_logic;                    -- 1 = invert (active-low fan)
    period    : in  unsigned(31 downto 0);        -- cycles per PWM period
    duty      : in  unsigned(31 downto 0);        -- high cycles (0..period)
    pwm_out   : out std_logic;
    raw_level : out std_logic
  );
end;
architecture rtl of pwm_core is
  signal cnt : unsigned(31 downto 0) := (others=>'0');
  signal raw : std_logic := '0';
begin
  process(clk)
  begin
    if rising_edge(clk) then
      if rst_n = '0' or enable = '0' then
        cnt <= (others=>'0');
        raw <= '0';
      else
        if cnt + 1 >= period then
          cnt <= (others=>'0');
        else
          cnt <= cnt + 1;
        end if;
        if cnt < duty then raw <= '1'; else raw <= '0'; end if;
      end if;
    end if;
  end process;
  raw_level <= raw;
  pwm_out   <= raw xor polarity;
end;
```

3. BOOT.BIN para AMD EDF

El .bit por si solo no prepara CPU, DDR, relojes y puentes. Se exporto hardware/XSA actualizado y se construyo xilinx-bootbin con herramientas EDF/Yocto/SDT. El DT base no incluye perifericos PL; un overlay Linux agrega reloj, interconnect y PWM. El bitstream ya viene dentro de BOOT.BIN; overlay final sin firmware-name evita recargarlo.

BIF que enumera los componentes

Fuente completa: hardware/fan-control-stage2/BOOT_fan_pwm_2026-10-24.bif

```
the_ROM_image:
{
  [bootloader, destination_cpu=a53-0] fsbl.elf
  [destination_cpu=pmu] pmufw.elf
  [destination_device=pl] download-axu3eg-ps.bit
  [destination_cpu=a53-0, exception_level=el-3, trustzone] arm-trusted-firmware.elf
  [destination_cpu=a53-0, load=0x100000] cortexa53-linux.dtb
  [destination_cpu=a53-0, exception_level=el-2] u-boot.elf
  [udf_bh] bootbin-version-header-axu3eg-ps.txt
}
```

- bitbake xilinx-bootbin termino correctamente; 99 warnings no fatales de tareas diferidas.

- SHA256 del bitstream empacado: 6220c077b5b2e3c6ee10243e7197332193f04ba7d1726eb1613e393b8d751d25; coincide con fuente Vivado/SDT.

- SHA256 del psu_init.c: aa824df4de5f74de6c0f884094bef2f5836847671f3c85bd1978839aad83ff0b; coincide con SDT.

- SHA256 del BOOT.BIN instalado: 9a41c75cf524e77d2d7aa4d97711f02b906d3af7fd0d9a4d6c06608938a910d9.

4. Lo que se descubrio en Linux

devmem ausente y /dev/mem

```
command -v devmem
ls -l /dev/mem
command -v busybox
busybox --list 2>/dev/null | grep '^devmem$'
id
command -v sudo
command -v python3
```

```
devmem: sin ruta; /dev/mem: crw-r----- 1 root kmem 1, 1; busybox/devmem: no aparece;
uid=1001(amd-edf); sudo=/usr/bin/sudo; python3=/usr/bin/python3
```

Una lectura Python/mmap directa de /dev/mem a 0xA0000000 termino con SSH 'Connection reset'. Ese texto no demuestra si Linux reinicio o solo se corto SSH; no repetimos el acceso crudo y pasamos a UIO.

CPU, memoria, disco, red

```
lscpu | grep -E 'Architecture|Model name|CPU\(s)'
free -h
lsblk -o NAME,SIZE,FSTYPE,MOUNTPOINTS
ip -br address
cat /proc/loadavg
cat /sys/class/fpga_manager/fpga0/state
```

```
Architecture: aarch64; CPU(s): 4; Model name: Cortex-A53
RAM: 3.8Gi total; lecturas compartidas ~285-300Mi usados
mmcblk0 58.3G: p1 1G /efi; p2 2G /storage; p3 55.3G /
mmcblk1 29.6G: eMMC detectada, sin montar
end0 UP: IP DHCP cambio .131, .132, .139 y finalmente .140
loadavg compartido 0.00 0.00 0.00; despues 0.51 0.39 0.16
fpga_manager: operating
```

La IP era DHCP, no estatica. La ultima observada fue 192.168.100.140/24. Las fechas de Linux/journal aparecieron a veces como enero de 2038; no son una buena prueba de cuando se cargo el bitstream.

Sensores AMS

```
for d in /sys/bus/iio/devices/iio:device*; do
  [ -e "$d" ] || continue
  echo "--- $d ---"
  cat "$d/name" 2>/dev/null
  for f in "$d"/*_label; do
    [ -r "$f" ] && printf '%s: %s\n' "${f##*/}" "$(cat "$f")"
  done
done
```

```
xilinx-ams
in_temp20_label: Temp_PL
in_temp7_label: Temp_LPD
in_temp8_label: Temp_FPD
/sys/class/hwmon/hwmon0/name: ams
temp1_label: Temp_LPD; temp2_label: Temp_FPD; temp3_label: Temp_PL
```

No habia thermal_zone visible. Las entradas hwmon son miligrados C; valores compartidos rondaron 27-30 C. El control toma la maxima de LPD, FPD y PL.

5. Overlay temporal y UIO

Primero se compilo un overlay exploratorio generic-uio en /tmp, solo para probar que el sistema podia exponer registros en la direccion vista por Vivado. Luego se preparo overlay final xlnx,axi-pwm-1.0 con reloj/puente, aplicado por el servicio.

```
dtc -I fs -O dts /proc/device-tree 2>/dev/null | grep -n -A12 -m1 'axi {'
# Se ve axi en el DT base, pero no estaba el PWM
command -v dtc
mount | grep configfs
modinfo uio_pdrv_genirq
sudo modprobe uio_pdrv_genirq of_id=generic-uio
```

```
cat > /tmp/fan-pwm-uio.dts <<'EOF'
/dts-v1/;
/plugin/;
/ { fragment@0 { target-path = "/axi"; __overlay__ {
 #address-cells = <2>; #size-cells = <2>;
 fan-pwm@a0000000 { compatible = "generic-uio";
 reg = <0x0 0xa0000000 0x0 0x10000>; status = "okay"; };
}; }; };
EOF
dtc -@ -I dts -O dtb -o /tmp/fan-pwm-uio.dtbo /tmp/fan-pwm-uio.dts
sudo sh -c 'mkdir /sys/kernel/config/device-tree/overlays/fan-pwm && cat /tmp/fan-pwm-uio.dtbo >
    /sys/kernel/config/device-tree/overlays/fan-pwm/dtbo'
```

```
find /proc/device-tree/axi -maxdepth 1 -iname '*fan*' -print
```

```
# /proc/device-tree/axi/fan-pwm@a0000000
# luego: uio0-uio3 axi-pmon; uio4 fan-pwm@a0000000
cat /sys/class/uio/uio4/maps/map0/addr
# 0x00000000a0000000
cat /sys/class/uio/uio4/maps/map0/size
# 0x0000000000010000
```

UIO (Userspace I/O) crea /dev/uioN identificado por el mapa del hardware. Se lee su map0/addr y se confirma que coincide A0000000. El overlay temporal fue exploracion; el setup final utiliza el compatible correcto y se vuelve a aplicar despues del reinicio.

6. Copia del arranque y verificacion por UART/SSH

```
# Windows
scp "Z:\AXU3EG_ORDENADO\AXU3EG_EDF_2026_1_VERSION2023_REFERENCIA\hardware\fan-control-stage2\BOOT_fan_pwm_
    2026-10-24.bin" amd-edf@192.168.100.139:/tmp/BOOT_fan_pwm.bin
ssh amd-edf@192.168.100.139

# Linux
sudo cp -a /efi/boot.bin /efi/boot.bin.pre-fan-pwm
ls -lh /efi/boot.bin /efi/boot.bin.pre-fan-pwm /tmp/BOOT_fan_pwm.bin
sudo cp /tmp/BOOT_fan_pwm.bin /efi/boot.bin
sync
sha256sum /efi/boot.bin
```

```
scp: 100%, 3605KB a 4.4MB/s
/efi/boot.bin viejo: 3.4M; copia pre-fan-pwm: 3.4M; archivo nuevo: 3.6M
SHA256: 9a41c75cf524e77d2d7aa4d97711f02b906d3af7fd0d9a4d6c06608938a910d9
UART volvio a mostrar login amd-edf; FPGA manager = operating
ip -br address: end0 UP 192.168.100.140/24
```

Se mantuvieron JTAG y UART conectados. La SD cargo el BOOT.BIN por si sola y Linux volvio. sync normalmente no imprime nada: fuerza escritura a la SD. Se guardo el arranque previo para poder revertir.

7. Instalacion de control en Linux

```
# PowerShell en Windows (IP actual de ejemplo)
$ip = "192.168.100.140"
$sw = "Z:\AXU3EG_ORDENADO\AXU3EG_EDF_2026_1_VERSION2023_REFERENCIA\hardware\fan-control-stage2\software\fa
    n-control"
scp "$sw\fan-pwm-control.py" "amd-edf@${ip}:/tmp/"
scp "$sw\fan-pwm-uio-setup.sh" "amd-edf@${ip}:/tmp/"
scp "$sw\fan-pwm-device-tree.dtso" "amd-edf@${ip}:/tmp/"
scp "$sw\fan-pwm.service" "amd-edf@${ip}:/tmp/"
```

```
# Linux por SSH o UART
sudo install -d /etc/fan-pwm /usr/local/sbin
sudo install -m 755 /tmp/fan-pwm-control.py /usr/local/sbin/fan-pwm-control.py
sudo install -m 755 /tmp/fan-pwm-uio-setup.sh /usr/local/sbin/fan-pwm-uio-setup.sh
sudo install -m 644 /tmp/fan-pwm-device-tree.dtso /etc/fan-pwm/fan-pwm-device-tree.dtso
sudo install -m 644 /tmp/fan-pwm.service /etc/systemd/system/fan-pwm.service
sudo systemctl daemon-reload
sudo systemctl enable --now fan-pwm.service
sudo systemctl status fan-pwm.service --no-pager
journalctl -u fan-pwm.service -n 20 --no-pager
```

install -m 755 instala ejecutables; 644 instala archivos de configuracion; daemon-reload recarga systemd; enable activa al arranque; --now inicia ya; status y journalctl consultan estado/log. La consola UART eco alguna linea con letras duplicadas; ls -l posterior confirmo los cuatro nombres correctos en sus destinos.

Salida del servicio

```
fan-pwm.service: loaded, enabled, active (running)
ExecStartPre fan-pwm-uio-setup.sh: status=0/SUCCESS
Main PID: 797 python3

Oct 24 21:35:50 Temp_LPD=28.6C Temp_FPD=28.5C Temp_PL=28.1C max=28.6C fan=0.0%
registers: CTRL=0x3 PERIOD=4000 DUTY=0
Oct 24 21:35:52 max=30.1C fan=0.0%; CTRL=0x3 PERIOD=4000 DUTY=0
Oct 24 21:35:54 max=30.2C fan=0.0%; CTRL=0x3 PERIOD=4000 DUTY=0
```

La fan quedo parada. Eso es prueba real de apagado, no que ya se valido el 30%. El primer controlador daba kick al iniciar el servicio, pero no necesariamente cuando la curva la encendiera despues. El archivo de GitHub se mejoro con kick al salir de apagado; aun no desplegado en placa.

8. Curva y codigo completo

Temperatura maxima	Orden
<38 C	Apagada (0%).
40 C	Enciende con impulso 100% 2 s; luego minimo 30%.
40-60 C	Rampa lineal 30%-100%.
>=60 C	100%.
Falla de sensores	100% fail-safe.
Ya encendida entre 38-40 C	Mantiene minimo; apaga a <=38 C (histeresis).

Periodo=4000 a PL0 ~99.999 MHz da ~25 kHz. 30% pide duty 1200. No existe tacometro en el cableado documentado; no se midieron RPM. Al parar limpiamente, el programa pide 100%. Sin watchdog independiente no hay garantia ante corte de energia o bloqueo brusco.

Controlador Python completo

Fuente completa: hardware/fan-control-stage2/software/fan-control/fan-pwm-control.py

```
#!/usr/bin/env python3
"""Temperature based fan control for AXU3EGB EDF Linux / AXI PWM IP."""
import glob
import mmap
import os
import struct
import sys
import time

PWM_BASE = 0xA0000000
PWM_PERIOD = 4000                 # 99.999 MHz / 4000 ~= 25 kHz
REG_CTRL, REG_PERIOD, REG_DUTY = 0x00, 0x04, 0x08
CTRL_ENABLE = 1 << 0
CTRL_POLARITY_INVERT = 1 << 1     # PWM output is active-low at FAN_PWM
START_C = 40.0
STOP_C = 38.0                     # hysteresis: don't chatter at 40 C
FULL_C = 60.0
MIN_RUNNING_PERCENT = 30.0        # must be confirmed against this fan
KICKSTART_SECONDS = 2.0
SAMPLE_SECONDS = 2.0


def log(message):
    print(f"fan-pwm: {message}", flush=True)


def read_temperatures():
    """Return labeled SoC temperatures in Celsius, gathered from hwmon."""
    values = []
    for label_path in glob.glob("/sys/class/hwmon/hwmon*/temp*_label"):
        try:
            with open(label_path, encoding="ascii") as f:
                label = f.read().strip()
            if label not in {"Temp_LPD", "Temp_FPD", "Temp_PL"}:
                continue
            input_path = label_path[:-len("_label")] + "_input"
            with open(input_path, encoding="ascii") as f:
                milli_c = int(f.read().strip())
            values.append((label, milli_c / 1000.0))
        except (OSError, ValueError) as exc:
            log(f"sensor read failed at {label_path}: {exc}")
    return values


def find_uio():
    """Find the UIO device whose map0 is the PWM's 0xA0000000 region."""
    for name_path in glob.glob("/sys/class/uio/uio*/name"):
        uio_dir = os.path.dirname(name_path)
        try:
            with open(os.path.join(uio_dir, "maps/map0/addr"), encoding="ascii") as f:
                addr = int(f.read().strip(), 0)
            with open(os.path.join(uio_dir, "maps/map0/size"), encoding="ascii") as f:
                size = int(f.read().strip(), 0)
            if addr == PWM_BASE and size >= 0x0C:
                return os.path.basename(uio_dir), size
        except (OSError, ValueError):
            continue
    raise RuntimeError("No UIO map found at 0xA0000000; fan stays at hardware reset state")


class Pwm:
    def __init__(self, uio_name, map_size):
```

```
        self.fd = os.open(f"/dev/{uio_name}", os.O_RDWR | os.O_SYNC)
        self.mem = mmap.mmap(self.fd, map_size, flags=mmap.MAP_SHARED,
                             prot=mmap.PROT_READ | mmap.PROT_WRITE, offset=0)

    def write(self, offset, value):
        struct.pack_into("<I", self.mem, offset, value & 0xFFFFFFFF)

    def set_percent(self, percent):
        percent = max(0.0, min(100.0, percent))
        duty = round(PWM_PERIOD * percent / 100.0)
        # Disable while changing timing registers; then enable active-low mode.
        self.write(REG_CTRL, 0)
        self.write(REG_PERIOD, PWM_PERIOD)
        self.write(REG_DUTY, duty)
        self.write(REG_CTRL, CTRL_ENABLE | CTRL_POLARITY_INVERT)
        return duty

    def close(self):
        self.mem.close()
        os.close(self.fd)


def desired_percent(temp_c, fan_running):
    """Off below 38 C; start at 40 C and ramp linearly to 100% at 60 C."""
    if temp_c <= STOP_C:
        return 0.0, False
    if not fan_running and temp_c < START_C:
        return 0.0, False
    if temp_c < START_C:
        return MIN_RUNNING_PERCENT, True
    if temp_c >= FULL_C:
        return 100.0, True
    span = (100.0 - MIN_RUNNING_PERCENT) / (FULL_C - START_C)
    return MIN_RUNNING_PERCENT + (temp_c - START_C) * span, True


def main():
    uio_name, map_size = find_uio()
    pwm = Pwm(uio_name, map_size)
    fan_running = True
    try:
        # Startup kick: briefly run full speed so the fan reliably starts.
        pwm.set_percent(100.0)
        log(f"{uio_name} mapped 0x{PWM_BASE:08X}; 25 kHz PWM; 100% kick for
    {KICKSTART_SECONDS:.0f}s")
        time.sleep(KICKSTART_SECONDS)
        while True:
            temperatures = read_temperatures()
            if not temperatures:
                # Sensor failure is fail-safe: request full fan, never turn it off.
                temp_c = None
                percent = 100.0
                fan_running = True
                log("no labeled SoC temperature sensors; fail-safe 100%")
            else:
                temp_c = max(value for _, value in temperatures)
                was_running = fan_running
                percent, should_run = desired_percent(temp_c, fan_running)
                if should_run and not was_running:
                    # Kick the fan when temperature control starts it from rest.
                    pwm.set_percent(100.0)
                    log(f"temperature reached {temp_c:.1f}C; 100% startup kick for
    {KICKSTART_SECONDS:.0f}s")
                    time.sleep(KICKSTART_SECONDS)
                    percent, fan_running = desired_percent(temp_c, True)
                else:
                    fan_running = should_run
                readings = ", ".join(f"{name}={value:.1f}C" for name, value in temperatures)
                log(f"{readings}; max={temp_c:.1f}C; fan={percent:.1f}%")
            duty = pwm.set_percent(percent)
            log(f"registers: CTRL=0x3 PERIOD={PWM_PERIOD} DUTY={duty}")
            time.sleep(SAMPLE_SECONDS)
    finally:
        # Leave the fan at full speed if the program is stopped cleanly.
        try:
            pwm.set_percent(100.0)
            log("stopping: left PWM at 100%")
        finally:
            pwm.close()


if __name__ == "__main__":
    try:
        main()
    except Exception as exc:
        log(f"fatal error: {exc}")
        sys.exit(1)
```

Overlay final DTSO

Fuente completa: hardware/fan-control-stage2/software/fan-control/fan-pwm-device-tree.dtso

```
/dts-v1/;
/plugin/;

&fpga_full {

        clocking0: clocking0 {
                clocks = <&zynqmp_clk 0x47>;
                clock-output-names = "fabric_clk";
                #clock-cells = <0x0>;
                assigned-clock-rates = <0x5f5dd19>;
                assigned-clocks = <&zynqmp_clk 0x47>;
                compatible = "xlnx,fclk";
        };

        afi0: afi0 {
                config-afi = <0x0 0x0 0x1 0x0 0x2 0x0 0x3 0x0 0x4 0x0 0x5 0x0 0x6 0x0 0x7 0x0 0x8
    0x0 0x9 0x0 0xa 0x0 0xb 0x0 0xc 0x0 0xd 0x0 0xe 0xa00 0xf 0x0>;
                resets = <&zynqmp_reset 0x74>,
                 <&zynqmp_reset 0x75>,
                 <&zynqmp_reset 0x76>,
                 <&zynqmp_reset 0x77>;
                compatible = "xlnx,afi-fpga";
        };
};

&amba {
        #address-cells = <0x2>;
        #size-cells = <0x2>;

        axi_pwm_0: axi_pwm@a0000000 {
                xlnx,name = "axi_pwm_0";
                clocks = <&zynqmp_clk 0x47>;
                reg = <0x0 0xa0000000 0x0 0x10000>;
                xlnx,edk-iptype = "PERIPHERAL";
                xlnx,ip-name = "axi_pwm";
                xlnx,s00-axi-addr-width = <0x4>;
                clock-names = "s00_axi_aclk";
                compatible = "xlnx,axi-pwm-1.0";
                xlnx,s00-axi-data-width = <0x20>;
        };
};
```

Instalador UIO

Fuente completa: hardware/fan-control-stage2/software/fan-control/fan-pwm-uio-setup.sh

```
#!/bin/sh
# Apply the Linux PL description, then bind the PWM registers through UIO.
# The BOOT.BIN already loaded the FPGA bitstream, so this overlay intentionally
# has no firmware-name property: it must not ask Linux to reload the bitstream.
set -eu

CONFIGFS=/sys/kernel/config
OVERLAYS="$CONFIGFS/device-tree/overlays"
OVERLAY_NAME=fan-pwm-platform
DTB=/run/fan-pwm-platform.dtbo
DTS=/etc/fan-pwm/fan-pwm-device-tree.dtso

if ! mountpoint -q "$CONFIGFS"; then
    mount -t configfs configfs "$CONFIGFS"
fi

has_pwm_uio() {
    for addr in /sys/class/uio/uio*/maps/map0/addr; do
        [ -r "$addr" ] || continue
        [ "$(cat "$addr")" = "0x00000000a0000000" ] && return 0
    done
    return 1
}

find_pwm_node() {
    for node in \
        /proc/device-tree/axi/axi_pwm@a0000000 \
        /proc/device-tree/amba_pl/axi_pwm@a0000000 \
        /proc/device-tree/amba/axi_pwm@a0000000; do
        if [ -d "$node" ]; then
            PWM_NODE=$node
            return 0
        fi
    done
    return 1
}

if has_pwm_uio; then
    exit 0
fi

if ! find_pwm_node; then
    # The base EDF Device Tree strips PL nodes; apply the generated PL overlay.
    [ -r "$DTS" ] || { echo "Missing PL overlay source: $DTS" >&2; exit 1; }
    dtc -@ -I dts -O dtb -o "$DTB" "$DTS"
    mkdir -p "$OVERLAYS/$OVERLAY_NAME"
```

```
    if ! cat "$DTB" > "$OVERLAYS/$OVERLAY_NAME/dtbo"; then
        rmdir "$OVERLAYS/$OVERLAY_NAME" 2>/dev/null || true
        echo "Could not apply the PL Device Tree overlay" >&2
        exit 1
    fi
    if ! find_pwm_node; then
        echo "Overlay applied but axi_pwm@a0000000 did not appear" >&2
        exit 1
    fi
fi

COMPAT=$(tr '\000' '\n' < "$PWM_NODE/compatible" | head -n 1)
[ -n "$COMPAT" ] || { echo "PWM node has no compatible string" >&2; exit 1; }
modprobe uio_pdrv_genirq "of_id=$COMPAT"

i=0
while [ "$i" -lt 50 ]; do
    has_pwm_uio && exit 0
    i=$((i + 1))
    sleep 0.1
done

echo "No UIO map found at 0xA0000000" >&2
exit 1
```

Unidad systemd

Fuente completa: hardware/fan-control-stage2/software/fan-control/fan-pwm.service

```
[Unit]
Description=AXU3EGB temperature-controlled fan PWM
After=local-fs.target
Before=multi-user.target

[Service]
Type=simple
ExecStartPre=/usr/local/sbin/fan-pwm-uio-setup.sh
ExecStart=/usr/bin/python3 /usr/local/sbin/fan-pwm-control.py
Restart=on-failure
RestartSec=3
User=root
Group=root

[Install]
WantedBy=multi-user.target
```

9. Comandos para observar y recuperar

```
ip -br address
cat /sys/class/fpga_manager/fpga0/state
sudo systemctl status fan-pwm.service --no-pager
journalctl -u fan-pwm.service -n 30 --no-pager
journalctl -fu fan-pwm.service   # seguimiento en vivo
free -h
lsblk -o NAME,SIZE,FSTYPE,MOUNTPOINTS

# detener y desactivar
sudo systemctl disable --now fan-pwm.service
# reactivar ahora y en futuros arranques
sudo systemctl enable --now fan-pwm.service
# restaurar BOOT.BIN anterior
sudo cp -a /efi/boot.bin.pre-fan-pwm /efi/boot.bin
sync
```

systemctl --failed mostro una espera de red wait-online fallida, pero end0 subio y SSH funciono; no prueba fallo del PWM. No se escribio la eMMC.

10. GitHub y pendientes

Se subieron 89 archivos del escenario PWM a main en el commit 6edb76d; el hash remoto coincidio. El bundle ocupa cerca de 14.7 MB.

- Incluye BOOT.BIN, BIF/manifest/log, XSA, proyecto Vivado fuente, constraints, IP AXI PWM, salidas SDT, overlay, scripts, servicio y Python.

- Se excluyeron caches/runs de Vivado, WIC, licencias y credenciales.

- El token que aparecio en el chat no se uso; Windows uso su gestor de credenciales. Revocalo y crea uno nuevo por seguridad.

```
Repositorio: abelwu2001/AXU3EGB_EDF-AMD_VIVADO-2026.2_testing
Rama main | Commit 6edb76d
BOOT.BIN SHA256 9a41c75cf524e77d2d7aa4d97711f02b906d3af7fd0d9a4d6c06608938a910d9
```

Siguiente prueba	Resultado que buscamos
Desplegar Python actualizado	Ver en log el kick de transicion y confirmar version.
Probar 30%	Fan arranca sola y no se detiene; ajustar minimo si no.
Probar 40-60 C cuidadosamente	Kick, rampa y 100% al llegar a 60 C.
Luego pantalla AN970	Proyecto separado; pantalla aun no esta en PL.

Glosario como loro

Termino	Explicacion corta
.bit	Configuracion de logica FPGA/PL.
BOOT.BIN	Paquete con componentes que arrancan PS y PL.
Constraint	Regla de pin/electrica/timing para Vivado.
AXI-Lite	Bus sencillo para escribir/leer registros.
Device Tree overlay	Descripcion de PL que Linux agrega al arbol base.
UIO	Puente kernel-usuario para mapear registros identificados.
DUTY	Porcentaje de tiempo activa, no RPM.
Histeresis	Umbrales distintos para no oscilar prendiendo/apagando.
systemd	Administra servicio al inicio de Linux.

Las respuestas de terminal se transcriben de lo que se pego en la conversacion. No hay grabacion de todas las pulsaciones ni de cada pantalla. Donde no habia salida exacta, se da un resumen identificado como tal, sin inventar texto.